ACS19955 ECG Project

Dataset Preparation, Verification, Preprocessing and Data Management

---

1. Purpose of This Document

This document explains **everything we have done to prepare the ACS19955 ECG dataset for model development**.

The goal is simple:

Take the original ACS19955 dataset, understand exactly what is inside it, verify that the ECG signals and clinical information are correct, create patient-safe dataset splits, preprocess the ECG signals in a reproducible way, and save them in a format that can be loaded quickly for model training.

At the end of this process, we should have a clean and well-organized dataset that can be directly given to the deep-learning model.

This document is also meant to prevent confusion later.

A researcher should be able to read this README and understand:

- Where the original data is stored
- What the `.csv`, `.hea`, `.dat`, and `.med` files are
- What the labels mean
- How train/validation/test were created
- How the ECG signals were processed
- What is inside the `.h5` files
- Why two ECGs were removed
- Which files are final
- Which things are now frozen and should not be changed

---

2. The Dataset We Started With

The project uses the **ACS19955 dataset**.

The dataset contains:

19,955 ECG recordings

The original dataset is divided into:

- Training metadata: **17,960 ECGs**
- Test metadata: **1,995 ECGs**

The test set is provided without clinical labels.

The dataset contains both:

1. ECG waveform data
2. Clinical/diagnostic metadata

These two parts are connected using record identifiers.

---

3. Where Everything Is Stored

The main project directory is:

```
/content/drive/MyDrive/ECG_Project
```

Inside it, the important structure is:

```
ECG_Project/
│
├── ACS19955/
│   └── raw/
│       ├── CSV.zip
│       ├── ECG_median_data.zip
│       └── ECG_row_data.zip
│
└── results/
    │
    ├── dataset_split/
    │   ├── train_split.csv
    │   ├── validation_split.csv
    │   ├── test_split.csv
    │   └── split_info.txt
    │
    └── preprocessing/
        └── processed/
            ├── train_fixed.h5
            ├── validation.h5
            ├── test.h5
            │
            ├── train_failures.csv
            ├── train_repaired.csv
```

```
└── train_before_id_repair.h5
```

The important idea is:

Raw data is never modified.

The original ZIP files remain in the `raw` directory.

All processed data is stored separately under `results`.

---

4. What Was Inside the Original Dataset?

The raw directory contained three ZIP files.

```
CSV.zip
ECG_median_data.zip
ECG_row_data.zip
```

We audited all three before doing any preprocessing.

4.1 CSV.zip

This ZIP contains:

```
CSV/
├── train.csv
└── test.csv
```

These CSV files contain the dataset metadata.

Training CSV

Shape:

```
17960 × 28
```

The training CSV contains:

Identification information

```
Patient_id
```

```
ecg_row_record
ecg_med_record
```

Patient information

```
gender
age
Time_Interval
```

Clinical labels

```
AMI
OMI
CTO
PCI
NSTEMI
STEMI
UA
LM
PLAD
MLAD
DLAD
DB
PLCX
MLCX
DLCX
OM
PRCA
MRCA
DRCA
VF_VT
Paced
Prior_PCI
```

There are therefore:

22 potential clinical target columns.

---

5. Understanding the ECG Files

The waveform information is stored in two different forms.

The important row-data files are:

```
XXXXXX.hea
XXXXXX.dat
```

For example:

```
00001.hea
00001.dat
```

There is also a `.med` representation.

---

6. What Is a `.hea` File?

The `.hea` file is the **header**.

It is a text file that describes the ECG recording.

For example, a header begins with information such as:

```
16558 12 500 5000
```

This tells us that the recording has:

- 12 leads
- 500 Hz sampling frequency
- 5000 samples

The remaining header lines describe the individual leads and their signal representation.

The header therefore tells software such as WFDB:

"Here is how the corresponding `.dat` file should be interpreted."

---

7. What Is a `.dat` File?

The `.dat` file contains the actual ECG waveform data.

It is a **binary file**.

We do not normally open it manually.

Instead, a library such as WFDB reads the `.hea` and `.dat` files together and converts the binary information into numerical ECG samples.

After loading with WFDB, the signal has the form:

```
(5000, 12)
```

Meaning:

```
5000 time samples
×
12 ECG leads
```

---

8. What Is a `.med` File?

The dataset also contains:

```
00001.med
00002.med
...
```

These files represent the ECG data in the dataset's machine/native format.

For our model pipeline, we use the standardized:

```
.hea + .dat
```

waveform representation because it can be read directly using WFDB and converted into numerical arrays.

The `.med` files are therefore retained as part of the original raw dataset but are **not the primary waveform source for our deep-learning preprocessing pipeline**.

---

9. Basic ECG Specifications

After auditing the waveform files, the recordings were found to follow the expected structure.

Each ECG contains:

Number of leads

```
12
```

The lead order is:

```
I
II
III
aVR
aVL
aVF
```

```
V1
V2
V3
V4
V5
V6
```

Sampling frequency

```
500 Hz
```

Number of samples

```
5000
```

Therefore:

```
500 samples/second × 10 seconds
= 5000 samples
```

So every ECG represents:

```
10 seconds
```

of 12-lead ECG data.

---

10. Metadata Audit

Before touching the ECG signals, we audited the metadata.

The original training and test sets were checked for:

- Missing values
- Data types
- Unique identifiers
- Duplicate records
- Patient overlap
- Record overlap
- Label distributions
- Dataset dimensions

10.1 Missing Values

Training:

```
No missing values
```

Test:

```
No missing values
```

This means the metadata itself did not contain missing entries.

---

11. Patient-Level Separation

One of the most important checks was patient overlap.

The original training and test sets contained:

```
Train patients: 17018
Test patients : 1891
```

Patients appearing in both:

```
0
```

Therefore:

There is no patient overlap between the original training and test sets.

This is important because the same patient's ECG must not appear in both training and testing.

Otherwise, the model could partially memorize the patient rather than learn ECG patterns.

---

12. Record-Level Separation

We also checked ECG record identifiers.

For:

```
ecg_row_record
```

the overlap between training and test was:

```
0
```

For:

```
ecg_med_record
```

the overlap was also:

```
0
```

Therefore:

There is no record overlap between the original training and test sets.

---

13. Understanding the Labels

The training data contains 22 clinical variables.

Some are binary:

```
0 / 1
```

while some vessel-related labels use:

```
0 / 1 / 2
```

The vessel labels represent the number of affected vessels according to the dataset's clinical definitions.

Examples include:

```
PLAD
MLAD
DLAD

PLCX
MLCX
DLCX

PRCA
MRCA
DRCA
```

The dataset also contains higher-level clinical conditions such as:

```
AMI
```

```
OMI
NSTEMI
STEMI
UA
CTO
```

and additional information such as:

```
PCI
Paced
Prior_PCI
VF_VT
```

---

14. Our Intended Prediction Problem

During project planning, we decided that we **will not treat "normal ECG" as an explicit target class**.

Instead, the model will focus on clinically meaningful abnormalities.

The intended prediction system should eventually answer questions such as:

Acute coronary syndrome

```
STEMI
NSTEMI
UA
```

Occlusion

```
OMI
```

Chronic total occlusion

```
CTO
```

Vessel/location information

If OMI or relevant coronary disease is present, the system should eventually identify the affected coronary territory/artery using the available labels.

This is a **multi-target clinical prediction problem**, rather than a simple single-class classification problem.

The exact model architecture and final target mapping will be decided during the model-development stage.

15. Creating the Validation Set

The original dataset provided:

```
17,960 training ECGs
```

We needed a separate validation set.

Instead of randomly splitting individual ECGs without considering patients, we created a **patient-safe split**.

The original test set was kept untouched.

The training portion was divided into:

```
Training
Validation
```

The final split is:

```
Train      : 15,275 ECGs
Validation :  2,685 ECGs
Test       :  1,995 ECGs
```

Total:

```
15,275 + 2,685 + 1,995
= 19,955 ECGs
```

The split files were saved here:

```
results/dataset_split/
```

Specifically:

```
train_split.csv
validation_split.csv
test_split.csv
split_info.txt
```

---

16. The Splits Are Now Fixed

This is extremely important.

The dataset split has already been created and verified.

Therefore:

We should not regenerate the train/validation/test split for individual experiments.

Every future model experiment should use the same split.

This gives us fair and reproducible comparisons between models.

If Model A performs better than Model B, we know that both models were evaluated on the same data.

---

17. Waveform Verification

Before preprocessing the complete dataset, we tested actual ECG waveform decoding.

We used WFDB to read sample records.

The checks included:

- Signal shape
- Sampling frequency
- Lead names
- Lead order
- NaN values
- Infinite values
- Signal amplitude
- Number of samples

The valid records produced:

```
Shape = (5000, 12)
Sampling frequency = 500 Hz
```

and the lead order was consistently:

```
I, II, III, aVR, aVL, aVF,
V1, V2, V3, V4, V5, V6
```

This confirmed that the waveform files could be decoded correctly.

---

18. Designing the Preprocessing Pipeline

Once the raw waveform format was verified, we defined a fixed preprocessing pipeline.

The pipeline is:

```
Raw ECG
   ↓
Signal validation
   ↓
Sampling-rate check
   ↓
Resampling if necessary
   ↓
Exactly 5000 samples
   ↓
0.5 Hz high-pass filter
   ↓
50 Hz low-pass filter
   ↓
50 Hz notch filter
   ↓
Z-score normalization
   ↓
Transpose
   ↓
Final tensor
   ↓
HDF5 storage
```

---

19. Signal Validation

Before processing an ECG, we check that:

- The signal is two-dimensional
- There are exactly 12 leads
- The signal contains finite numerical values
- The ECG can be decoded successfully

If an ECG cannot be safely processed, it is not silently used.

It is recorded in a failure log.

---

20. Sampling Frequency

The target sampling frequency is:

```
500 Hz
```

If a recording has a different sampling frequency, the pipeline resamples it to 500 Hz.

The goal is that all model inputs have the same time resolution.

---

21. Signal Length

The final signal length is:

```
5000 samples
```

At 500 Hz:

```
5000 / 500
= 10 seconds
```

Therefore every final ECG has exactly:

```
10 seconds
```

of signal.

---

22. Filtering

Three filtering operations were used.

High-pass filter

```
0.5 Hz
```

Purpose:

Reduce very slow baseline drift.

Low-pass filter

```
50 Hz
```

Purpose:

Reduce higher-frequency noise.

Notch filter

```
50 Hz
```

Purpose:

Reduce 50 Hz electrical interference.

The final preprocessing description stored in the H5 metadata is:

```
0.5Hz_HP + 50Hz_LP + 50Hz_notch
```

---

23. Normalization

After filtering, each ECG is normalized using **segment-level Z-score normalization**.

The signal is transformed using:

```
(signal - mean) / standard deviation
```

The mean and standard deviation are calculated from the individual ECG segment.

The purpose is to put the signals onto a consistent numerical scale while preserving their waveform structure.

After normalization, the resulting ECG has approximately:

```
mean ≈ 0
standard deviation ≈ 1
```

---

24. Final Tensor Format

Before preprocessing, WFDB gives us:

```
(5000, 12)
```

That means:

```
Time × Leads
```

Deep-learning models will use:

```
(12, 5000)
```

That means:

```
Leads × Time
```

Therefore one ECG becomes:

```
(12, 5000)
```

For a batch of 20 ECGs:

```
(20, 12, 5000)
```

For the complete training set:

```
(15273, 12, 5000)
```

---

25. Why HDF5 Was Used

Processing thousands of WFDB files every time a model starts would be inefficient.

Instead, we preprocess the ECGs once and store the processed signals in HDF5 (`.h5`) files.

This gives us a much simpler workflow:

```
Original WFDB files
        ↓
Preprocess once
        ↓
H5 files
        ↓
Fast model loading
```

The model does not need to repeatedly decode the original `.dat` files.

---

26. What Is Inside an H5 File?

Each H5 file contains two main datasets.

For example:

```
train_fixed.h5
```

contains:

```
signals
record_ids
```

26.1 `signals`

The training file contains:

```
Shape:
(15273, 12, 5000)
```

This means:

```
15273 ECG recordings
×
12 leads
×
5000 samples
```

The data type is:

```
float32
```

So each ECG is already converted into a numerical array ready for a neural network.

---

27. Understanding One Entry in `signals`

Suppose we access:

```
signals[0]
```

We get:

```
(12, 5000)
```

This is one ECG.

The first dimension contains the 12 leads:

```
0  → I
1  → II
2  → III
3  → aVR
4  → aVL
5  → aVF
6  → V1
7  → V2
8  → V3
9  → V4
10 → V5
11 → V6
```

Each lead contains:

```
5000 samples
```

---

28. What Is `record_ids`?

The second dataset is:

```
record_ids
```

For training:

```
15273 IDs
```

For validation:

```
2685 IDs
```

For testing:

```
1995 IDs
```

These IDs allow us to connect the processed ECG signal back to the metadata.

For example:

```
record_id
    ↓
H5 signal
    ↓
metadata CSV
    ↓
clinical labels
```

This is extremely important.

The H5 file stores the waveform.

The CSV stores the clinical information.

The record ID connects them.

---

29. H5 Attributes

The H5 files also contain descriptive attributes.

For example:

```
sampling_frequency: 500
duration_seconds: 10
num_leads: 12
num_samples: 5000
lead_order:
I, II, III, aVR, aVL, aVF, V1, V2, V3, V4, V5, V6

normalization:
segment_zscore

preprocessing:
0.5Hz_HP + 50Hz_LP + 50Hz_notch
```

These attributes document how the signals were prepared.

This means someone opening the H5 file later can understand what the numbers represent.

---

30. The Corrupted ECG Records

During full-dataset preprocessing, five problematic training records were initially detected.

They were:

```
16558
14262
3054
2008
3228
```

Further investigation showed that the dataset uses zero-padded filenames for some records.

For example:

```
3054
```

was actually stored as:

```
03054
```

Similarly:

```
2008  → 02008
3228  → 03228
```

Therefore, three apparent file-not-found problems were actually filename-mapping issues.

---

31. Actual Signal Problems

After correcting the filename mapping, two records still could not be loaded correctly:

```
14262
03228
```

WFDB reported:

```
Samples were not loaded correctly
```

These two recordings were therefore treated as unusable.

We did **not** invent signal values for them.

We did **not** silently replace them.

We did **not** force them into the training dataset.

They were excluded from the final processed training H5.

---

32. Another Three Records With NaN

Three other records initially contained one NaN value each.

The records were:

```
16558
03054
02008
```

The NaN values were detected during validation.

These were handled by the preprocessing/repair procedure and recorded in:

```
train_repaired.csv
```

The repaired records were successfully included in the final training H5.

This is documented rather than hidden.

---

33. Final Training Dataset

The original training split contained:

```
15,275 ECGs
```

Two ECGs were unusable:

```
14262
03228
```

Therefore:

```
15,275 - 2
= 15,273 usable ECGs
```

The final training H5 is:

```
train_fixed.h5
```

with:

```
Signals:
(15273, 12, 5000)

Record IDs:
15273
```

---

34. Training H5 Repair

Before removing the two unusable records, the original processed H5 was backed up.

Backup:

```
results/preprocessing/processed/
train_before_id_repair.h5
```

The final repaired file is:

```
results/preprocessing/processed/
train_fixed.h5
```

The repair was verified.

Results:

```
Expected usable ECGs : 15273
Actual H5 ECGs       : 15273
Missing IDs          : 0
Extra IDs            : 0
Duplicate IDs        : 0
```

All signal values were finite.

The signal shape was correct.

Therefore the repaired training H5 is the final training waveform file.

---

35. Final Dataset Sizes

The current final dataset is:

Split	ECGs	Patients
Training	15,273	derived from fixed split
Validation	2,685	2,553
Test	1,995	1,891
Total usable	**19,953**	—

The original dataset contained:

```
19,955 ECGs
```

Two training ECGs were excluded because they could not be decoded correctly.

Therefore:

```
19,955 original
-
2 unusable
=
19,953 usable
```

---

36. Final H5 Files

Training

```
/content/drive/MyDrive/ECG_Project/results/preprocessing/processed/train_fixe
d.h5
```

Shape:

```
(15273, 12, 5000)
```

Validation

```
/content/drive/MyDrive/ECG_Project/results/preprocessing/processed/validation
.h5
```

Shape:

```
(2685, 12, 5000)
```

Test

```
/content/drive/MyDrive/ECG_Project/results/preprocessing/processed/test.h5
```

Shape:

```
(1995, 12, 5000)
```

---

37. Important Difference Between CSV and H5

This distinction should always be remembered.

CSV

The CSV contains:

```
Patient information
+
Record IDs
+
Clinical labels
```

For example:

```
Patient_id
age
gender
AMI
OMI
CTO
STEMI
NSTEMI
UA
...
```

H5

The H5 contains:

```
Processed ECG waveform
+
Record ID
```

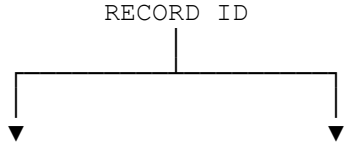
For example:

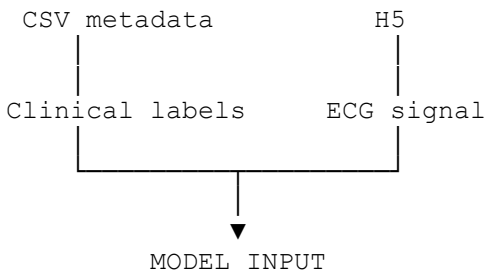
```
signals
record_ids
```

The H5 does not need to duplicate all clinical metadata.

We connect the two using the record ID.

Conceptually:





---

38. What the Model Will Eventually Receive

The deep-learning model will not receive:

```
.dat
.hea
.med
```

directly.

Instead, it will receive the processed numerical ECG.

For one ECG:

```
(12, 5000)
```

For a batch:

```
(B, 12, 5000)
```

where `B` is the batch size.

For example:

```
(32, 12, 5000)
```

means:

```
32 ECGs
×
12 leads
×
5000 samples
```

---

39. What Has Been Verified So Far

The following parts have been checked.

Raw dataset

- Raw ZIP files exist
- ZIP contents were inspected
- Expected waveform files were found

Metadata

- Train shape verified
- Test shape verified
- Missing values checked
- Data types checked
- Record identifiers checked
- Patient overlap checked
- Record overlap checked
- Label distributions audited

Waveforms

- WFDB decoding tested
- 12 leads verified
- Lead order verified
- 500 Hz verified
- 5000 samples verified
- Signal dimensions verified
- Non-finite values investigated

Splitting

- Training split created
- Validation split created
- Test split preserved
- Patient-safe separation checked
- Split files saved

Preprocessing

- Signal validation implemented
- Sampling-rate handling implemented
- Length standardized
- 0.5 Hz high-pass filtering applied
- 50 Hz low-pass filtering applied
- 50 Hz notch filtering applied
- Segment Z-score normalization applied

- Final shape converted to `(12, 5000)`

Storage

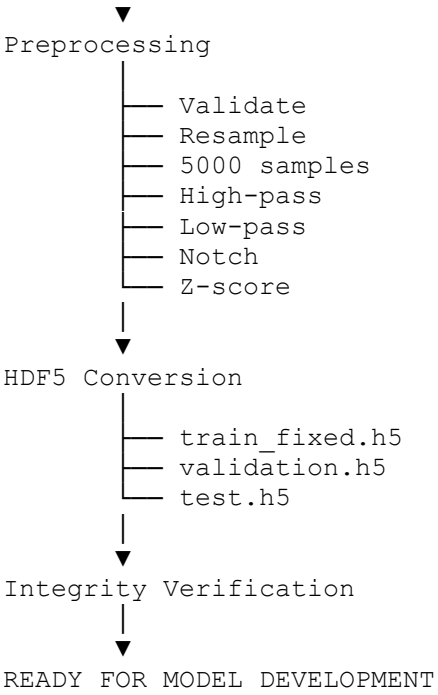
- HDF5 files created
- Signals stored as `float32`
- Record IDs stored
- Processing attributes stored
- Training H5 repaired
- Final IDs verified
- Signal shapes verified
- Finite-value checks passed

---

40. Current Project Data Flow

The entire journey can now be understood as one simple story.

```
ACS19955 Dataset
        │
        ▼
Raw ZIP files
        │
        ├── CSV.zip
        ├── ECG_median_data.zip
        └── ECG_row_data.zip
        │
        ▼
Dataset Audit
        │
        ├── Metadata checked
        ├── Labels checked
        ├── Patients checked
        └── Records checked
        │
        ▼
Waveform Audit
        │
        ├── .hea verified
        ├── .dat verified
        ├── WFDB decoding verified
        └── Lead order verified
        │
        ▼
Patient-Safe Split
        │
        ├── Train
        ├── Validation
        └── Test
        │
```



---

41. What We Should NOT Do From This Point

The preprocessing stage should now be treated as **frozen**.

We should not:

- Randomly regenerate the train/validation/test split
- Change the lead order
- Change the sampling frequency
- Change the signal length
- Apply different preprocessing to validation or test
- Modify the test set
- Reintroduce the two unusable ECGs
- Randomly delete additional ECGs
- Change normalization halfway through experiments
- Process some experiments differently without documenting it
- Replace the final H5 files with unverified versions

If a future experiment requires a different preprocessing strategy, it should be treated as a **new preprocessing experiment**, not as a silent modification of the current dataset.

---

42. Why This Matters for Research

The whole purpose of doing this carefully is reproducibility.

Suppose later we train:

```
Model A
Model B
Model C
ECGFounder
Our proposed model
```

All of them should receive the same underlying dataset.

Then the comparison is meaningful.

For example:

```
Same train split
Same validation split
Same test split
Same ECG preprocessing
Same labels
```

Only then can we confidently say:

Model A performed better than Model B.

---

43. The Two Excluded ECGs

For complete transparency, the final dataset excludes:

```
14262
03228
```

Reason:

```
WFDB:
Samples were not loaded correctly
```

These records were not safely recoverable through the current waveform decoding pipeline.

They are therefore excluded from the processed training data.

This exclusion is documented and should be mentioned in any future dataset-processing documentation or paper.

---

44. Current Final State

At this exact stage, the project has moved from:

```
Raw Dataset
```

to:

```
Verified + Split + Preprocessed Dataset
```

The final usable ECG counts are:

```
TRAIN:
15,273

VALIDATION:
2,685

TEST:
1,995
```

Total usable:

```
19,953 ECGs
```

Every processed ECG has:

```
12 leads
500 Hz
5000 samples
10 seconds
float32
```

and is stored as:

```
(12, 5000)
```

inside the H5 files.

---

45. The Most Important Concept to Remember

The entire dataset can now be thought of in only three layers.

Layer 1 — Original data

```
CSV
.hea
.dat
.med
```

This is the source material.

Layer 2 — Processed ECG

```
H5
```

This contains the clean numerical ECG signals that the model can use.

Layer 3 — Clinical information

```
CSV metadata
```

This tells us what clinical condition/label belongs to each ECG.

The record ID connects Layer 2 and Layer 3.

Therefore:

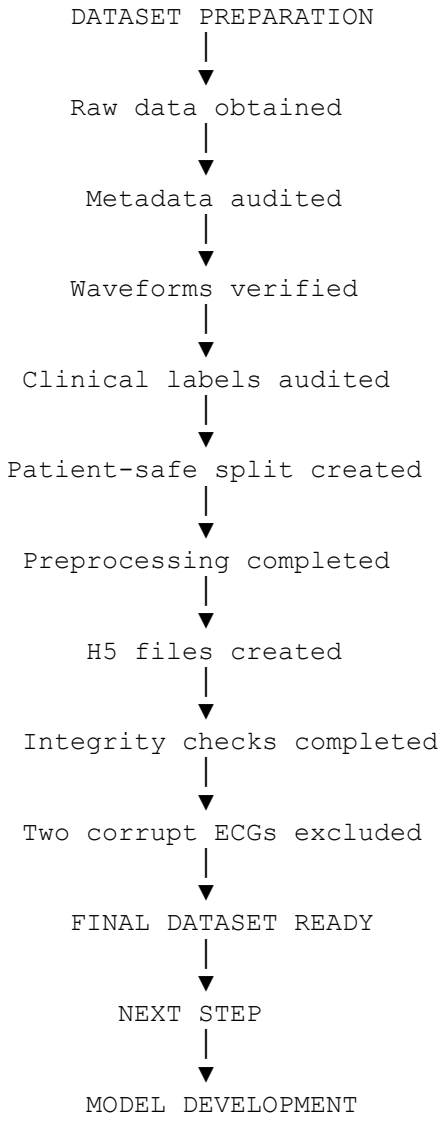
```
ECG signal
    +
record ID
    +
clinical labels
    ↓
training example
```

That is the basic structure of our entire machine-learning dataset.

---

46. Where We Stand Before Model Building

We are now at the following point:



We should therefore consider the **dataset preparation and preprocessing stage complete**.

The next stage should begin only after this README and the final integrity checks are fully understood.

---

47. Final File Reference

Raw data

```
/content/drive/MyDrive/ECG_Project/ACS19955/raw/
```

Contains:

```
CSV.zip
ECG_median_data.zip
ECG_row_data.zip
```

Fixed dataset splits

```
/content/drive/MyDrive/ECG_Project/results/dataset_split/
```

Contains:

```
train_split.csv
validation_split.csv
test_split.csv
split_info.txt
```

Final processed ECGs

```
/content/drive/MyDrive/ECG_Project/results/preprocessing/processed/
```

Contains:

```
train_fixed.h5
validation.h5
test.h5
```

Processing logs

```
train_failures.csv
train_repaired.csv
```

Backup

```
train_before_id_repair.h5
```

---

48. One-Sentence Summary

We started with 19,955 raw ACS19955 ECG recordings, audited the metadata and waveform files, created a patient-safe train/validation/test split, standardized and verified the ECG signals, preprocessed them into 12-lead 10-second `(12,5000)` tensors, stored them

efficiently in HDF5, and excluded only two training recordings that could not be decoded correctly—leaving 19,953 usable ECGs ready for model development.

STATUS

```
DATASET AUDIT                ✓ COMPLETE
METADATA AUDIT               ✓ COMPLETE
WAVEFORM VERIFICATION        ✓ COMPLETE
LABEL AUDIT                  ✓ COMPLETE
PATIENT-SAFE SPLIT           ✓ COMPLETE
PREPROCESSING                ✓ COMPLETE
HDF5 STORAGE                 ✓ COMPLETE
CORRUPTED RECORD HANDLING    ✓ COMPLETE
H5 INTEGRITY CHECK           ✓ COMPLETE

================================================
DATA PREPARATION COMPLETE
================================================

NEXT STAGE:
MODEL DEVELOPMENT
```